



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

ОТЧЕТ

по лабораторной работе № 8, 9

по курсу «Защита информации»

на тему: «Создание электронной подписи. Шифрование и расшифровка
произвольных и архивных файлов»

Студент ИУ7-71Б
(Группа)

(Подпись, дата)

Я. Р. Овчинников
(И. О. Фамилия)

Преподаватель

(Подпись, дата)

Ю. С. Руденкова
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

ЗАДАНИЕ	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
РЕАЛИЗАЦИЯ	19
ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ	26

ЗАДАНИЕ

Цель работы

Создание электронной подписи. Шифрование и расшифровка произвольного файла и архивного файла.

Задание

Реализовать программу создания и проверки электронной подписи для документа. Обеспечить шифрование и расшифровку произвольного файла с использованием разработанной программы. Предусмотреть работу программы с пустым, однобайтовым файлом. Программа также должна уметь обрабатывать файл архива (rar, zip или др.).

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Вопросы из лабораторной 8

1. Дайте определение электронной цифровой подписи

Электронная цифровая подпись (ЭЦП) — это криптографическое преобразование набора данных, которое позволяет получателю данных убедиться в их целостности и подлинности, а также защищает от отказа отправителя от авторства.

ЭЦП обеспечивает три основные функции безопасности:

1.1. Аутентификация

Подтверждение авторства документа. Подпись может быть создана только обладателем секретного (приватного) ключа, что гарантирует подлинность отправителя.

1.2. Целостность данных

Любое изменение в подписанном документе делает подпись недействительной. Даже изменение одного бита в данных приводит к несовпадению вычисленного хэша с расшифрованным из подписи.

1.3. Неотказуемость (non-repudiation)

Отправитель не может отрицать факт создания подписи, так как только он обладает приватным ключом, необходимым для её формирования.

1.4. Принцип работы ЭЦП

Процесс создания и проверки ЭЦП включает следующие этапы:

Создание подписи:

- 1) Вычисляется хэш-функция от документа: $H = \text{hash}(M)$
- 2) Хэш шифруется приватным ключом отправителя: $S = E_{\text{priv}}(H)$
- 3) Подпись S прикрепляется к документу M

Проверка подписи:

- 1) Получатель вычисляет хэш документа: $H_1 = \text{hash}(M)$
- 2) Расшифровывает подпись публичным ключом отправителя: $H_2 = D_{\text{pub}}(S)$
- 3) Сравнивает хэши: если $H_1 = H_2$, подпись верна

Основные свойства:

- Подпись создаётся с использованием приватного ключа
- Проверка выполняется с использованием публичного ключа
- Невозможно создать подпись без знания приватного ключа
- Подпись уникальна для каждого документа

2. Опишите алгоритм создания цифровой подписи

Алгоритм создания цифровой подписи состоит из нескольких этапов:

2.1. Генерация ключевой пары

Первый этап — создание пары криптографических ключей (для асимметричного шифрования):

Для RSA:

- 1) Выбираются два больших простых числа p и q
- 2) Вычисляется модуль: $n = p \times q$
- 3) Вычисляется функция Эйлера: $\phi(n) = (p - 1)(q - 1)$
- 4) Выбирается публичная экспонента e (обычно 65537), взаимно простая с $\phi(n)$
- 5) Вычисляется приватная экспонента d : $d \equiv e^{-1} \pmod{\phi(n)}$
- 6) Публичный ключ: (n, e)
- 7) Приватный ключ: (n, d) (также хранятся p, q)

2.2. Вычисление хэш-функции документа

Для создания подписи необходимо вычислить хэш-функцию от исходного документа:

$$H = \text{hash}(M)$$

где M — исходное сообщение (документ), H — хэш-значение фиксированной длины.

Требования к хэш-функции:

- Детерминированность (один и тот же вход даёт одинаковый хэш)
- Быстрое вычисление
- Необратимость (невозможно восстановить исходные данные по хэшу)
- Стойкость к коллизиям (сложно найти два разных документа с одинаковым хэшем)
- Лавинный эффект (малое изменение входа сильно меняет хэш)

Примеры хэш-функций:

- SHA-256 (256-битный хэш)
- SHA-3 (Кескак)
- MD5 (устаревший, небезопасный)
- DJB2 (простая, быстрая, используется в данной работе для демонстрации)

2.3. Шифрование хэша приватным ключом

Полученный хэш-значение шифруется приватным ключом владельца:

$$S = H^d \bmod n$$

где S — цифровая подпись, d — приватная экспонента, n — модуль RSA. Это преобразование может выполнить только владелец приватного ключа.

2.4. Прикрепление подписи к документу

Подпись S прикрепляется к исходному документу M . Возможны два варианта:

Вариант 1: Подпись с сообщением

- Формат файла: $[_][][][_]$
- Размер файла увеличивается на размер подписи
- Этот подход используется в данной работе

Вариант 2: Отдельная подпись

- Подпись хранится в отдельном файле
- Исходный файл не изменяется
- Требуется передача двух файлов

2.5. Алгоритм в виде псевдокода

Алгоритм создания ЭЦП:

Вход: документ M , приватный ключ (n, d)

Выход: подписанный документ (M, S)

```
1.  $H = \text{compute\_hash}(M)$  // Хэширование документа
2.  $H\_mod = H \bmod n$  // Приведение к диапазону модуля
3.  $S = \text{mod\_pow}(H\_mod, d, n)$  // Шифрование хэша
4.  $\text{output} = [\text{size}(S)] [S] [M]$  // Формирование выходного файла
5.  $\text{return output}$ 
```

В данной работе для хэширования используется алгоритм DJB2, а для создания подписи — RSA с 64-битным модулем.

3. Опишите алгоритм проверки цифровой подписи

Проверка цифровой подписи — это процесс, позволяющий убедиться в подлинности и целостности документа.

3.1. Получение подписанного документа

Получатель получает файл, содержащий:

- Размер подписи (4 байта)
- Цифровую подпись S (8 байт)
- Исходный документ M

Формат: $[sig_size][signature][original_data]$

3.2. Извлечение компонентов

Из полученного файла извлекаются:

- 1) Размер подписи для валидации формата
- 2) Подпись S
- 3) Исходные данные документа M

Проверяется корректность формата файла (минимальный размер, корректность размера подписи).

3.3. Вычисление хэша оригинальных данных

Получатель вычисляет хэш извлечённых данных:

$$H_1 = hash(M)$$

Используется та же хэш-функция, что и при создании подписи (в данной работе — DJB2).

3.4. Расшифровка подписи публичным ключом

Подпись расшифровывается с использованием публичного ключа отправителя:

$$H_2 = S^e \bmod n$$

где e — публичная экспонента, n — модуль RSA.

Это даёт хэш-значение, которое было зашифровано при создании подписи.

3.5. Сравнение хэшей

Сравниваются два хэш-значения:

- H_1 — вычисленный хэш полученных данных
- H_2 — расшифрованный хэш из подписи

Если $H_1 = H_2$:

- Подпись **верна**
- Документ не был изменён
- Подпись создана владельцем приватного ключа
- Оригинальный файл восстанавливается и сохраняется

Если $H_1 \neq H_2$:

- Подпись **неверна**
- Возможные причины:
 - Документ был изменён после подписания
 - Подпись повреждена
 - Подпись создана другим ключом
 - Использована неправильная хэш-функция
- Файл не сохраняется (компрометирован)

3.6. Алгоритм в виде псевдокода

Алгоритм проверки ЭЦП:

Вход: подписанный документ, публичный ключ (n , e)

Выход: статус проверки (`valid/invalid`)

1. Извлечь из файла:

```
sig_size = read_uint32(file)
```

```

S = read_uint64(file)
M = read_remaining_data(file)

2. Проверить формат:
   if sig_size != 8 then return INVALID

3. H1 = compute_hash(M)           // Вычислить хэш данных
4. H1_mod = H1 mod n              // Привести к диапазону модуля

5. H2 = mod_pow(S, e, n)          // Расшифровать подпись

6. if H1_mod == H2 then
    save_file(M)                  // Сохранить оригинал
    return VALID                  // Подпись верна
else
    return INVALID                // Подпись неверна

```

В данной работе при неверной подписи файл не сохраняется для предотвращения использования скомпрометированных данных.

Вопросы из лабораторной 9

1. Дайте определение электронной цифровой подписи

Данный вопрос был подробно рассмотрен в Блоке 1, пункт 1.

2. Дайте определение электронного сертификата

Электронный сертификат (цифровой сертификат) — это электронный документ, который связывает публичный ключ с идентификационными данными его владельца и подтверждается цифровой подписью доверенного центра сертификации (Certification Authority, CA).

2.1. Структура электронного сертификата

Сертификат обычно соответствует стандарту X.509 и содержит:

Основные поля:

- **Версия** — версия стандарта X.509 (обычно v3)
- **Серийный номер** — уникальный идентификатор сертификата
- **Алгоритм подписи** — используемый алгоритм (RSA, ECDSA и др.)
- **Издатель (Issuer)** — имя центра сертификации, выдавшего сертификат
- **Срок действия** — период валидности (Not Before / Not After)
- **Субъект (Subject)** — данные владельца сертификата
 - CN (Common Name) — доменное имя или ФИО
 - O (Organization) — организация
 - OU (Organizational Unit) — подразделение
 - C (Country) — страна
 - E (Email) — электронная почта
- **Публичный ключ субъекта** — публичный ключ владельца
- **Цифровая подпись СА** — подпись центра сертификации

Расширения (Extensions):

- Key Usage — назначение ключа (подпись, шифрование и т.д.)
- Extended Key Usage — расширенное использование (SSL-сервер, email и др.)
- Subject Alternative Name — альтернативные имена
- CRL Distribution Points — точки распространения списков отзыва

2.2. Назначение электронного сертификата

Сертификат обеспечивает следующие функции:

1. Аутентификация владельца:

- Подтверждает, что публичный ключ принадлежит конкретному лицу или организации

- Предотвращает атаки типа «человек посередине» (MITM)

2. Доверие к публичному ключу:

- Центр сертификации проверяет владельца перед выдачей сертификата
- Подпись СА гарантирует подлинность связи ключа с владельцем

3. Управление жизненным циклом ключей:

- Контроль срока действия ключей
- Возможность отзыва скомпрометированных сертификатов

2.3. Виды сертификатов

По назначению:

- SSL/TLS сертификаты — для защиты веб-сайтов (HTTPS)
- Code Signing — для подписи программного обеспечения
- Email сертификаты (S/MIME) — для защиты электронной почты
- Client сертификаты — для аутентификации пользователей

По уровню проверки (для SSL):

- DV (Domain Validation) — проверяется только владение доменом
- OV (Organization Validation) — проверяется организация
- EV (Extended Validation) — расширенная проверка, зелёная строка в браузере

2.4. Цепочка доверия

Сертификаты образуют иерархическую цепочку доверия:

Корневой СА (Root)

Промежуточный СА (Intermediate CA)

Конечный сертификат (End-Entity Certificate)

Принцип работы:

- Корневой СА самоподписанный, встроен в операционную систему или браузер
- Промежуточный СА подписан корневым СА
- Конечный сертификат подписан промежуточным СА
- При проверке строится цепочка до доверенного корневого СА

Отзыв сертификатов:

- CRL (Certificate Revocation List) — список отозванных сертификатов
- OCSP (Online Certificate Status Protocol) — онлайн-проверка статуса

В данной работе не реализуется полноценная инфраструктура PKI и выдача сертификатов, однако используются ключевые принципы асимметричной криптографии: генерация пары ключей (публичный/приватный), подпись приватным ключом и проверка публичным ключом.

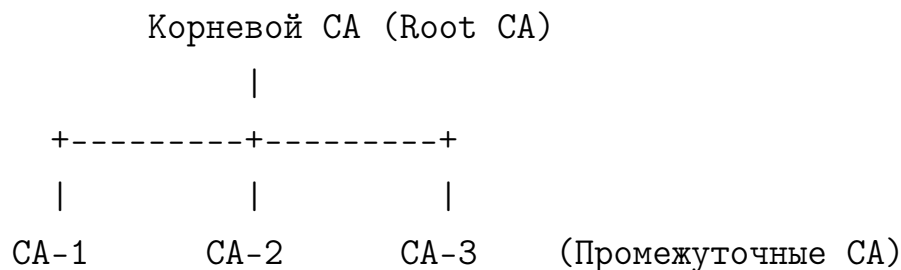
3. Какие существуют модели организации инфраструктуры электронных сертификатов?

Инфраструктура открытых ключей (PKI — Public Key Infrastructure) — это комплекс программно-аппаратных средств, организационных мер и процедур для управления жизненным циклом электронных сертификатов и ключей.

Существует несколько моделей организации PKI:

3.1. Иерархическая модель (Hierarchical PKI)

Структура:



cert1 cert2 cert3 (Конечные сертификаты)

Характеристики:

- Древовидная иерархия центров сертификации
- Корневой СА — единственный источник доверия
- Промежуточные СА делегируют полномочия вниз по иерархии
- Строгая централизация управления

Преимущества:

- Простота управления и администрирования
- Ясная цепочка доверия
- Легко масштабируется вертикально
- Высокая безопасность корневого СА (может быть offline)

Недостатки:

- Единая точка отказа (компрометация Root СА критична)
- Сложность горизонтального масштабирования
- Жёсткая структура подчинённости

Применение:

- Корпоративные PKI
- Государственные инфраструктуры
- Военные и закрытые системы

3.2. Сетевая модель (Mesh/Network PKI)

Структура:

CA-1 <> CA-2

CA-3 <> CA-4

Все CA связаны между собой перекрёстными сертификатами

Характеристики:

- Все CA находятся на одном уровне (peer-to-peer)
- Отсутствует единый корневой центр
- CA взаимно сертифицируют друг друга (кросс-сертификация)
- Децентрализованная модель доверия

Преимущества:

- Высокая отказоустойчивость (нет единой точки отказа)
- Гибкость в организации доверия
- Возможность сотрудничества независимых организаций

Недостатки:

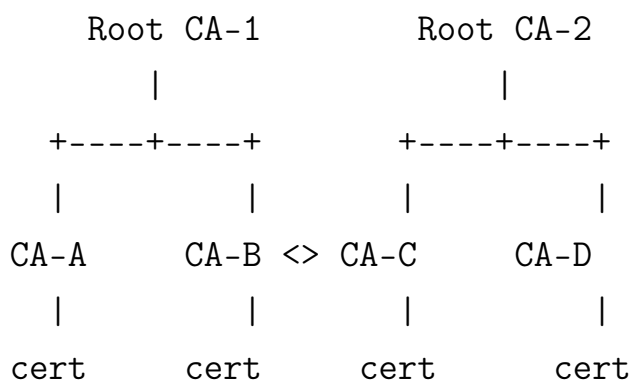
- Сложность управления и администрирования
- Необходимость поддерживать множество доверительных отношений
- Сложность построения и проверки цепочек доверия
- Масштабируемость: $O(n^2)$ связей для n центров

Применение:

- Федеративные системы (несколько независимых организаций)
- Межкорпоративное взаимодействие
- Системы B2B (business-to-business)

3.3. Гибридная модель (Hybrid PKI)

Структура:



Иерархия внутри доменов + кросс-сертификация между доменами

Характеристики:

- Комбинация иерархической и сетевой моделей
- Иерархия внутри каждой организации
- Кросс-сертификация между организациями
- Гибкость в настройке доверительных отношений

Преимущества:

- Баланс между простотой управления и отказоустойчивостью
- Автономность внутри организаций
- Возможность межорганизационного взаимодействия
- Оптимальна для крупных распределённых систем

Недостатки:

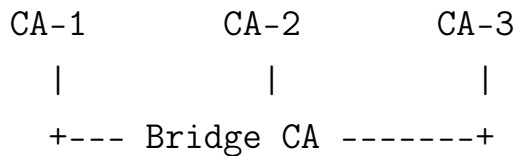
- Повышенная сложность по сравнению с чисто иерархической моделью
- Требуется координация политик безопасности между организациями

Применение:

- Крупные корпорации с филиалами
- Государственные системы с региональными подразделениями
- Отраслевые решения (банковский сектор, здравоохранение)

3.4. Модель моста (Bridge CA Model)

Структура:



Bridge CA связывает независимые иерархии

Характеристики:

- Специальный промежуточный CA (Bridge CA) соединяет независимые PKI
- Каждая организация сохраняет свою иерархию
- Bridge CA выполняет роль «посредника» доверия

Преимущества:

- Упрощение кросс-сертификации (вместо $O(n^2)$ связей — $O(n)$)
- Автономность участников
- Централизованная точка для межорганизационного доверия

Недостатки:

- Bridge CA становится критической точкой инфраструктуры
- Требуется согласования политик между всеми участниками

Применение:

- Федеральные PKI (например, Federal Bridge CA в США)
- Объединение корпоративных PKI в рамках альянсов

3.5. Сравнительная таблица моделей

Критерий	Иерархия	Сеть	Гибрид	Мост
Управление	Простое	Сложное	Средне	Средне
Отказоустойчивость	Низкая	Высокая	Средняя	Средняя
Масштабируемость	Вертикаль	Ограничена	Высокая	Высокая
Сложность цепочек	Низкая	Высокая	Средняя	Средняя
Централизация	Да	Нет	Частично	Частично

В данной работе реализованы базовые принципы асимметричной криптографии (генерация ключей, подпись, проверка), что является основой для построения любой из описанных моделей PKI.

РЕАЛИЗАЦИЯ

На листинге 1 приведена реализация функции генерации ключей RSA.

Листинг 1 – Генерация ключей RSA

```
1 void RSA::generate_keys() {
2     std::cout << "Генерация пары ключей RSA...\n";
3
4     // Генерация двух различных простых чисел
5     std::random_device rd;
6     std::mt19937_64 gen(rd());
7
8     p = generate_prime(10000, 50000);
9     do {
10         q = generate_prime(10000, 50000);
11     } while (q == p);
12
13     // Вычисление модуля n = p * q
14     n = p * q;
15
16     // Вычисление функции Эйлера (n) = (p-1)(q-1)
17     uint64_t phi = (p - 1) * (q - 1);
18
19     // Выбор публичной экспоненты e (обычно 65537)
20     e = 65537;
21     if (e >= phi || gcd(e, phi) != 1) {
22         // Если 65537 не подходит, ищем другое значение
23         for (e = 3; e < phi; e += 2) {
24             if (gcd(e, phi) == 1) {
25                 break;
26             }
27         }
28     }
29
30     // Вычисление приватной экспоненты d (обратное к e по модулю
31         (n))
32     d = mod_inverse(e, phi);
33
34     keys_generated = true;
35
36     std::cout << "Ключи успешно сгенерированы.\n";
```

```

36     std::cout << "Размер модуля: " << n << "\n\n";
37 }

```

Функция `generate_keys` генерирует пару ключей для алгоритма RSA. Сначала генерируются два различных простых числа p и q в диапазоне 10000-50000. Затем вычисляется модуль $n = p \times q$ и функция Эйлера $\phi(n) = (p - 1)(q - 1)$. Публичная экспонента устанавливается равной 65537 (стандартное значение), если она взаимно проста с $\phi(n)$. Приватная экспонента d вычисляется как обратное к e по модулю $\phi(n)$ с использованием расширенного алгоритма Евклида.

На листинге 2 приведена реализация функции создания электронной подписи.

Листинг 2 – Создание электронной подписи

```

1 void RSA::sign_file(const std::string& input_path, const
    std::string& output_path) {
2     if (!keys_generated) {
3         throw RSAExceptions::KeyNotGenerated();
4     }
5
6     // Чтение файла
7     std::vector<uint8_t> file_data = read_file(input_path);
8
9     // Вычисление хэша
10    uint32_t hash = compute_hash(file_data);
11
12    // Приведение хэша к диапазону модуля (чтобы hash < n)
13    uint64_t hash_mod = hash % n;
14
15    // Создание подписи (шифрование хэша приватным ключом)
16    uint64_t signature = mod_pow(hash_mod, d, n);
17
18    // Подготовка данных для записи
19    std::vector<uint8_t> output_data;
20
21    // Запись размера подписи (8 байт для uint64_t)
22    uint32_t sig_size = sizeof(signature);
23    output_data.insert(output_data.end(),
24                        reinterpret_cast<uint8_t*>(&sig_size),
25                        reinterpret_cast<uint8_t*>(&sig_size) +

```

```

26         sizeof(sig_size));
27     // Запись подписи
28     output_data.insert(output_data.end(),
29         reinterpret_cast<uint8_t*>(&signature),
30         reinterpret_cast<uint8_t*>(&signature) +
31             sizeof(signature));
32
33     // Запись оригинальных данных файла
34     output_data.insert(output_data.end(), file_data.begin(),
35         file_data.end());
36
37     // Сохранение подписанного файла
38     write_file(output_path, output_data);
39
40     std::cout << "Файл подписан успешно.\n";
41     std::cout << "Размер оригинала: " << file_data.size() << "
42         байт\n";
43     std::cout << "Размер с подписью: " << output_data.size() <<
44         " байт\n";
45 }

```

Функция `sign_file` создаёт электронную подпись для произвольного файла. Сначала файл полностью считывается в память. Затем вычисляется хэш-функция DJB2 от содержимого файла, получается 32-битное значение. Хэш приводится к диапазону модуля операцией остатка от деления. Подпись создаётся путём возведения хэша в степень d по модулю n (шифрование приватным ключом). Выходной файл формируется в формате: 4 байта размера подписи, 8 байт самой подписи, затем оригинальные данные файла.

На листинге 3 приведена реализация функции проверки электронной подписи.

Листинг 3 – Проверка электронной подписи

```

1 bool RSA::verify_file(const std::string& signed_path, const
2     std::string& output_path) {
3     if (!keys_generated) {
4         throw RSAExceptions::KeyNotGenerated();
5     }
6
7     // Чтение подписанного файла

```

```

7      std::vector<uint8_t> signed_data = read_file(signed_path);
8
9      // Проверка минимального размера
10     if (signed_data.size() < sizeof(uint32_t) +
11         sizeof(uint64_t)) {
12         throw RSAExceptions::InvalidSignatureFormat();
13     }
14
15     // Извлечение размера подписи
16     uint32_t sig_size;
17     std::copy(signed_data.begin(),
18               signed_data.begin() + sizeof(sig_size),
19               reinterpret_cast<uint8_t*>(&sig_size));
20
21     if (sig_size != sizeof(uint64_t)) {
22         throw RSAExceptions::InvalidSignatureFormat();
23     }
24
25     // Извлечение подписи
26     uint64_t signature;
27     std::copy(signed_data.begin() + sizeof(uint32_t),
28               signed_data.begin() + sizeof(uint32_t) +
29               sizeof(signature),
30               reinterpret_cast<uint8_t*>(&signature));
31
32     // Извлечение оригинальных данных
33     std::vector<uint8_t> original_data(
34         signed_data.begin() + sizeof(uint32_t) +
35         sizeof(uint64_t),
36         signed_data.end()
37     );
38
39     // Вычисление хэша оригинальных данных
40     uint32_t computed_hash = compute_hash(original_data);
41     uint64_t computed_hash_mod = computed_hash % n;
42
43     // Расшифровка подписи публичным ключом
44     uint64_t decrypted_hash = mod_pow(signature, e, n);
45
46     // Сравнение хэшей (после модуля)
47     bool is_valid = (computed_hash_mod == decrypted_hash);

```

```

45
46     if (is_valid) {
47         write_file(output_path, original_data);
48         std::cout << "Подпись ВЕРНА.\n";
49     } else {
50         std::cout << "Подпись НЕВЕРНА!\n";
51     }
52
53     return is_valid;
54 }

```

Функция `verify_file` проверяет подлинность электронной подписи. Подписанный файл считывается и проверяется корректность формата (минимальный размер, корректность размера подписи). Из файла извлекаются: размер подписи, сама подпись и оригинальные данные. Вычисляется хэш извлечённых данных. Подпись расшифровывается публичным ключом (возведение в степень e по модулю n), что даёт хэш-значение, которое было зашифровано при создании подписи. Если вычисленный хэш совпадает с расшифрованным, подпись считается верной и оригинальный файл сохраняется. В противном случае подпись признаётся недействительной.

На листинге 4 приведена реализация вспомогательных криптографических функций.

Листинг 4 – Вспомогательные функции RSA

```

1 // Модульное возведение в степень (быстрое возведение в степень)
2 uint64_t RSA::mod_pow(uint64_t base, uint64_t exp, uint64_t mod)
3     const {
4         uint64_t result = 1;
5         base = base % mod;
6
7         while (exp > 0) {
8             if (exp % 2 == 1) {
9                 result = (__uint128_t)result * base % mod;
10            }
11            exp = exp >> 1;
12            base = (__uint128_t)base * base % mod;
13        }
14
15        return result;

```

```

15 }
16
17 // Простая хэш-функция (DJB2)
18 uint32_t RSA::compute_hash(const std::vector<uint8_t>& data)
19     const {
20     uint32_t hash = 5381;
21
22     for (uint8_t byte : data) {
23         hash = ((hash << 5) + hash) + byte; // hash * 33 + byte
24     }
25
26     return hash;
27 }
28
29 // НОД (алгоритм Евклида)
30 uint64_t RSA::gcd(uint64_t a, uint64_t b) const {
31     while (b != 0) {
32         uint64_t temp = b;
33         b = a % b;
34         a = temp;
35     }
36     return a;
37 }
38
39 // Модульное обратное (расширенный алгоритм Евклида)
40 uint64_t RSA::mod_inverse(uint64_t a, uint64_t m) const {
41     int64_t m0 = m;
42     int64_t x0 = 0, x1 = 1;
43
44     if (m == 1) return 0;
45
46     while (a > 1) {
47         int64_t q = a / m;
48         int64_t t = m;
49
50         m = a % m;
51         a = t;
52         t = x0;
53
54         x0 = x1 - q * x0;
55         x1 = t;

```



```

55     }
56
57     if (x1 < 0) x1 += m0;
58
59     return x1;
60 }

```

Функция `mod_pow` выполняет быстрое модульное возведение в степень методом «возведения в квадрат и умножения». Используется 128-битная арифметика (`__uint128_t`) для предотвращения переполнения при умножении 64-битных чисел. Функция `compute_hash` реализует хэш-алгоритм DJB2 — простую, но эффективную хэш-функцию для демонстрационных целей. Функция `gcd` вычисляет наибольший общий делитель методом Евклида. Функция `mod_inverse` вычисляет обратное по модулю с использованием расширенного алгоритма Евклида, необходимое для вычисления приватной экспоненты `d`.

ДЕМОНСТРАЦИЯ РАБОТЫ ПРОГРАММЫ

На листинге ниже продемонстрирован основной сценарий работы программы:

Листинг 5 – Основной сценарий работы программы

```
1  lab8_9 git:(feature/lab8_9-rsa-signature) ./rsa_signature
2  === Электронная подпись RSA - Лабораторная работа 8-9 ===
3
4  === Электронная подпись RSA ===
5  1. Сгенерировать пару ключей
6  2. Загрузить ключи из файлов
7  3. Сохранить ключи в файлы
8  4. Создать электронную подпись для файла
9  5. Проверить электронную подпись файла
10 6. Показать ключи
11 0. Выход
12 > 6
13 Ключи еще не сгенерированы. Сначала создайте или загрузите ключи.
14
15 === Электронная подпись RSA ===
16 1. Сгенерировать пару ключей
17 2. Загрузить ключи из файлов
18 3. Сохранить ключи в файлы
19 4. Создать электронную подпись для файла
20 5. Проверить электронную подпись файла
21 6. Показать ключи
22 0. Выход
23 > 1
24
25 === Генерация пары ключей ===
26 Генерация пары ключей RSA...
27 Ключи успешно сгенерированы.
28 Размер модуля: 428472349
29
30
31 === Электронная подпись RSA ===
32 1. Сгенерировать пару ключей
33 2. Загрузить ключи из файлов
34 3. Сохранить ключи в файлы
35 4. Создать электронную подпись для файла
36 5. Проверить электронную подпись файла
```

```

37 6. Показать ключи
38 0. Выход
39 > 3
40
41 === Сохранение ключей ===
42 Введите базовое имя для файлов ключей (без расширения): my_key1
43 Публичный ключ сохранен: out/my_key1.pub
44 Приватный ключ сохранен: out/my_key1.priv
45
46
47 === Электронная подпись RSA ===
48 1. Сгенерировать пару ключей
49 2. Загрузить ключи из файлов
50 3. Сохранить ключи в файлы
51 4. Создать электронную подпись для файла
52 5. Проверить электронную подпись файла
53 6. Показать ключи
54 0. Выход
55 > 2
56
57 === Загрузка ключей ===
58 Введите имя файла публичного ключа (без пути, в директории
    out/): my_key1.pub
59 Введите имя файла приватного ключа (без пути, в директории
    out/): my_key1.priv
60 Ключи успешно загружены.
61 Публичный ключ: out/my_key1.pub
62 Приватный ключ: out/my_key1.priv
63
64
65 === Электронная подпись RSA ===
66 1. Сгенерировать пару ключей
67 2. Загрузить ключи из файлов
68 3. Сохранить ключи в файлы
69 4. Создать электронную подпись для файла
70 5. Проверить электронную подпись файла
71 6. Показать ключи
72 0. Выход
73 > 4
74
75 === Создание электронной подписи ===

```

```

76
77 Доступные файлы:
78     1. test.txt
79
80 Выберите файл для подписания (введите номер от 1 до 1, 0 для
    отмены): 1
81
82 Подписание: test.txt -> test.txt.signed
83 Файл подписан успешно.
84 Размер оригинала: 110 байт
85 Размер с подписью: 122 байт
86 Хэш файла: 0x57a9519d
87
88 Подписанный файл сохранен: out/test.txt.signed
89
90 === Электронная подпись RSA ===
91 1. Сгенерировать пару ключей
92 2. Загрузить ключи из файлов
93 3. Сохранить ключи в файлы
94 4. Создать электронную подпись для файла
95 5. Проверить электронную подпись файла
96 6. Показать ключи
97 0. Выход
98 > 5
99
100 === Проверка электронной подписи ===
101
102 Доступные файлы:
103     1. test.txt.signed
104
105 Выберите файл для проверки подписи (введите номер от 1 до 1, 0
    для отмены): 1
106
107 Проверка подписи: test.txt.signed
108 Подпись ВЕРНА.
109 Вычисленный хэш: 0x57a9519d
110 Хэш из подписи: 0x000000000b0b6946
111 Оригинальный файл восстановлен: out/verified_test.txt
112 Размер файла: 110 байт
113
114 === Электронная подпись RSA ===

```

```

115 1. Сгенерировать пару ключей
116 2. Загрузить ключи из файлов
117 3. Сохранить ключи в файлы
118 4. Создать электронную подпись для файла
119 5. Проверить электронную подпись файла
120 6. Показать ключи
121 0. Выход
122 > 6
123
124 === Параметры RSA ===
125 Модуль (n):          428472349
126 Публичная экспонента (e): 65537
127 Приватная экспонента (d): 219154353
128 Простое число p:      21179
129 Простое число q:      20231
130 Размер ключа (бит):    64
131
132 === Электронная подпись RSA ===
133 1. Сгенерировать пару ключей
134 2. Загрузить ключи из файлов
135 3. Сохранить ключи в файлы
136 4. Создать электронную подпись для файла
137 5. Проверить электронную подпись файла
138 6. Показать ключи
139 0. Выход
140 > 0
141 Выход из программы.

```

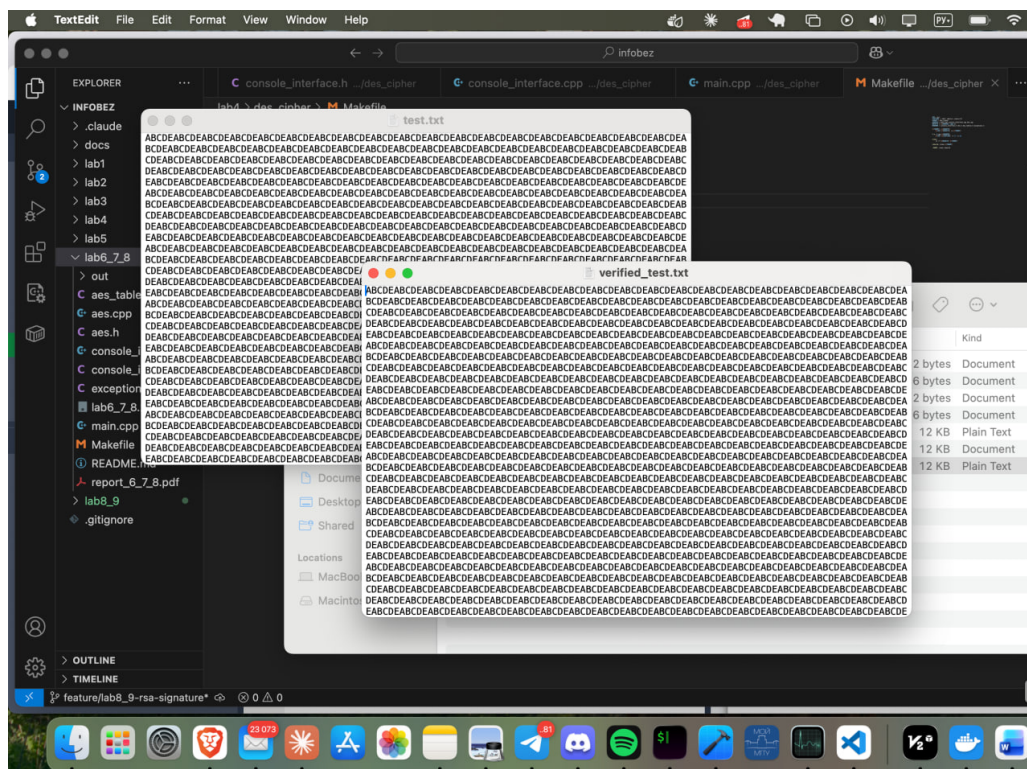


Рисунок 1 – Подпись большого файла

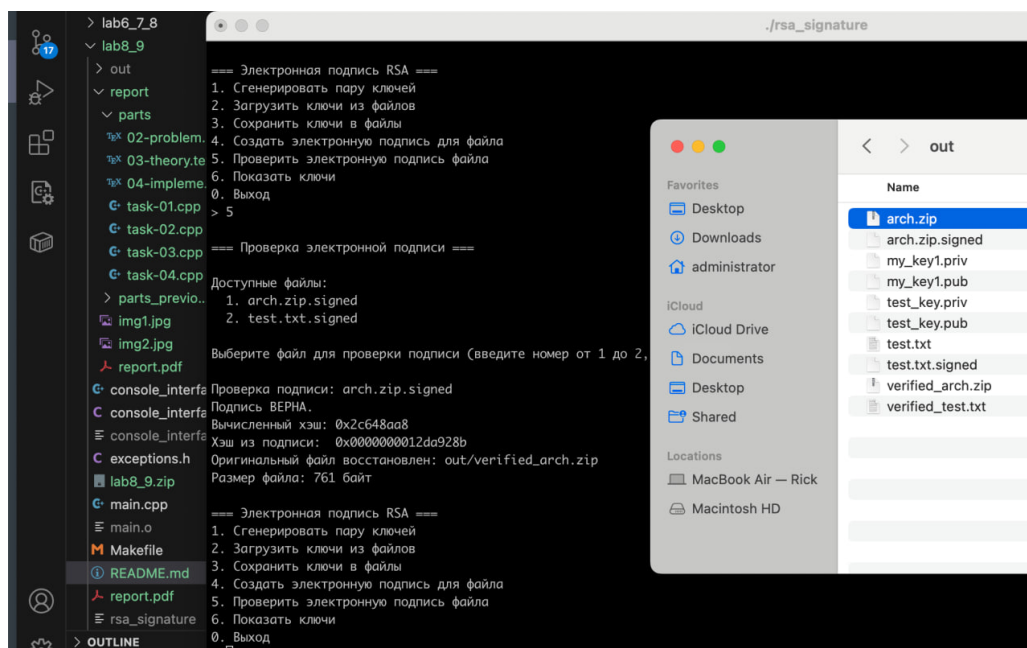


Рисунок 2 – Подпись архива

Также была предусмотрена обработка большинства ошибочных ситуаций без экстренного завершения программы:

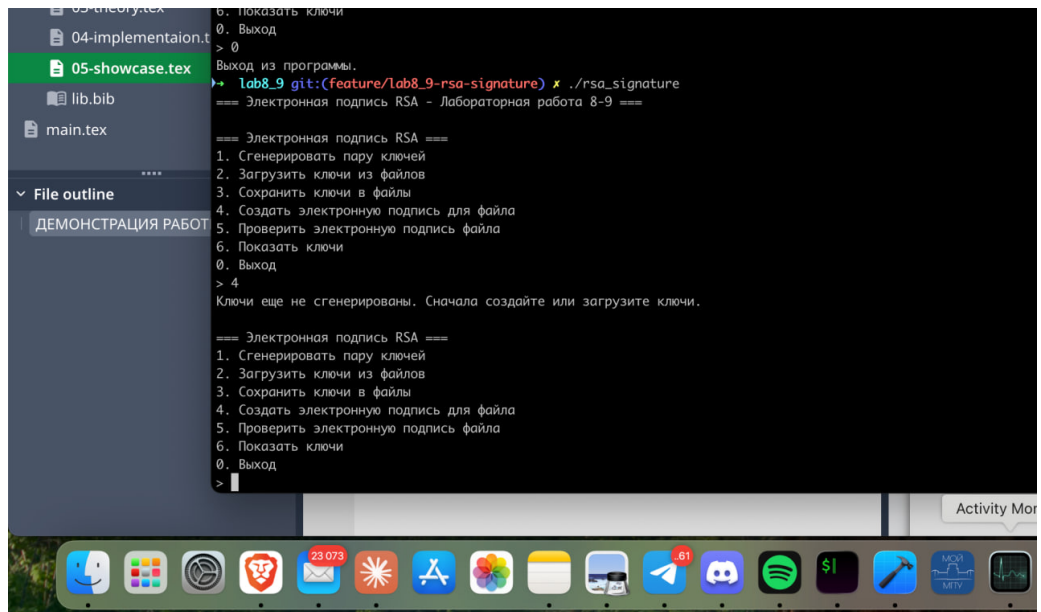


Рисунок 3 – Обработка ошибочных ситуаций